UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
DEPARTMAN ZA INDUSTRIJSKO INŽENJERSTVO I MENADŽMENT

Objedinjeni jezik modeliranja Unified Modeling Language (UML) II deo

dr Teodora Vučković,
vanredni profesor



Poslovni informacioni sistemi
2026/2027

Agenda

01 Statički dijagrami

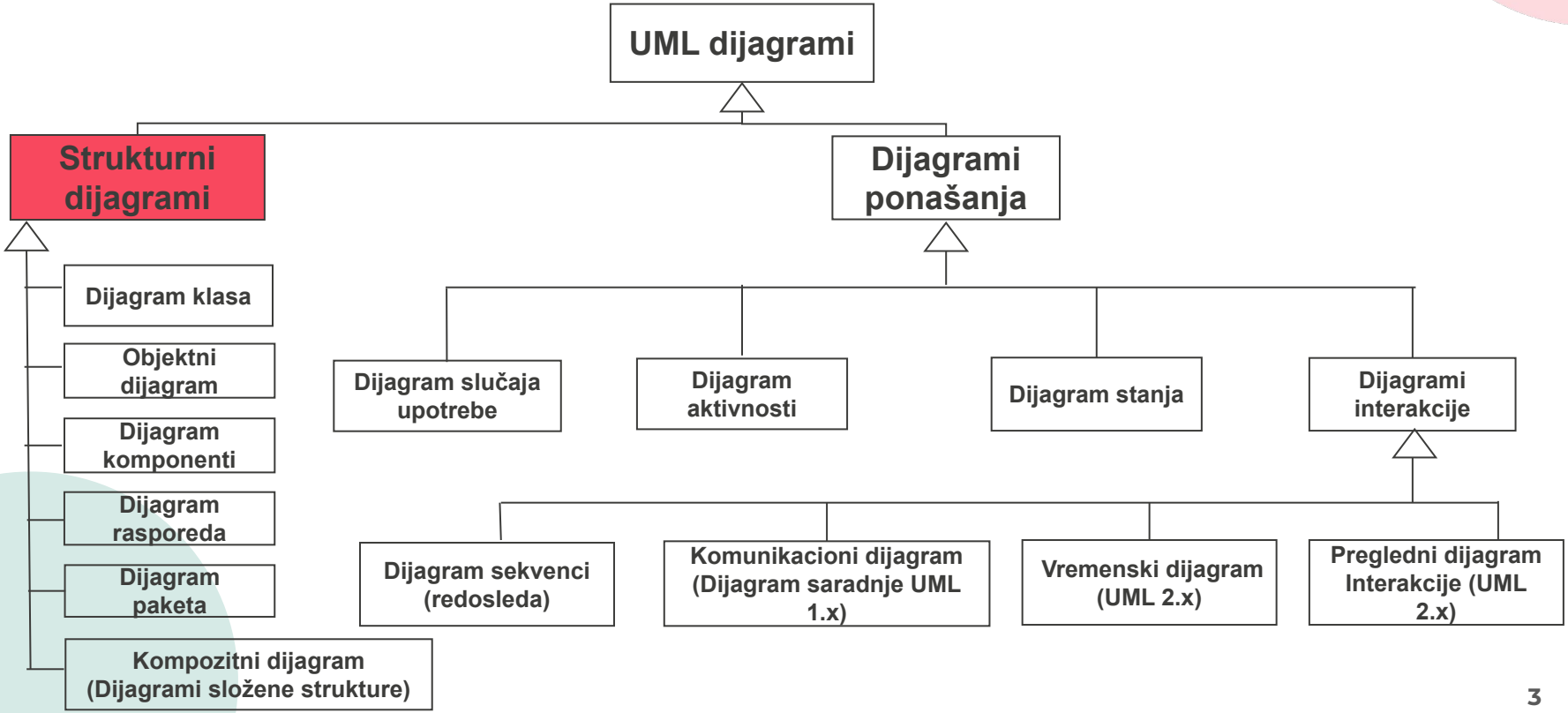
02 Dijagram klasa

03 Veza, navigacija, ograničenja

04 Poruka, Interfejs

05 Vežba

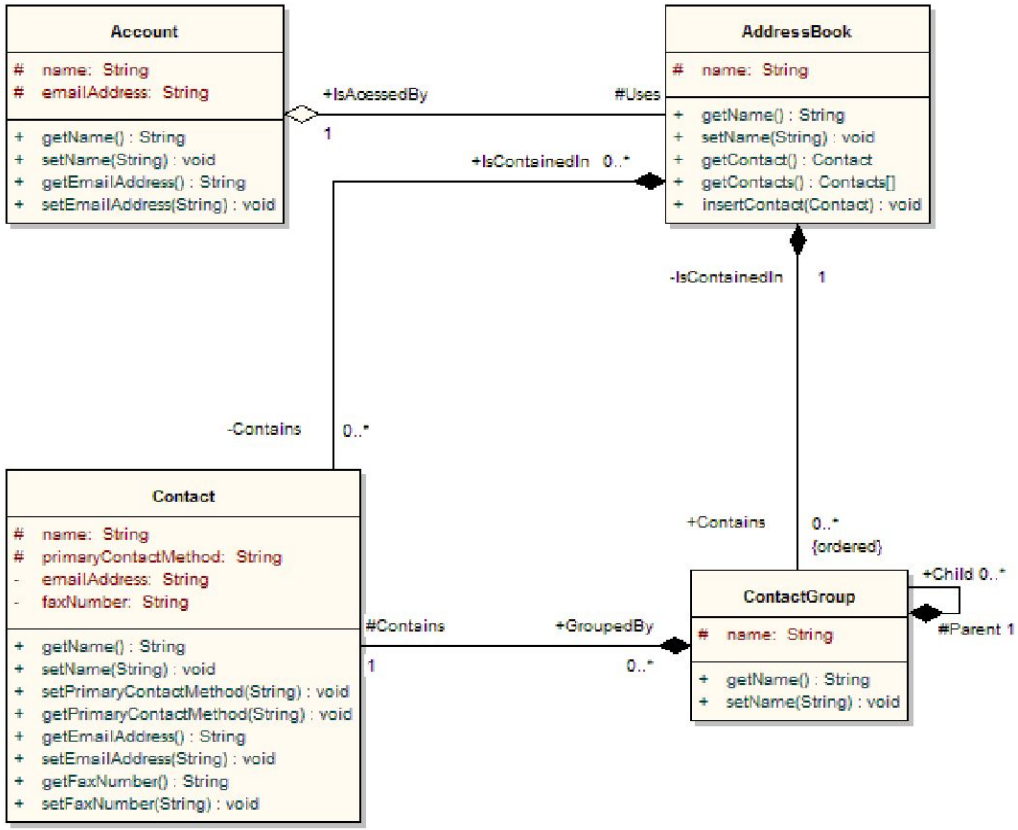
Koncepti funkcionalnog modelovanja ponašanja



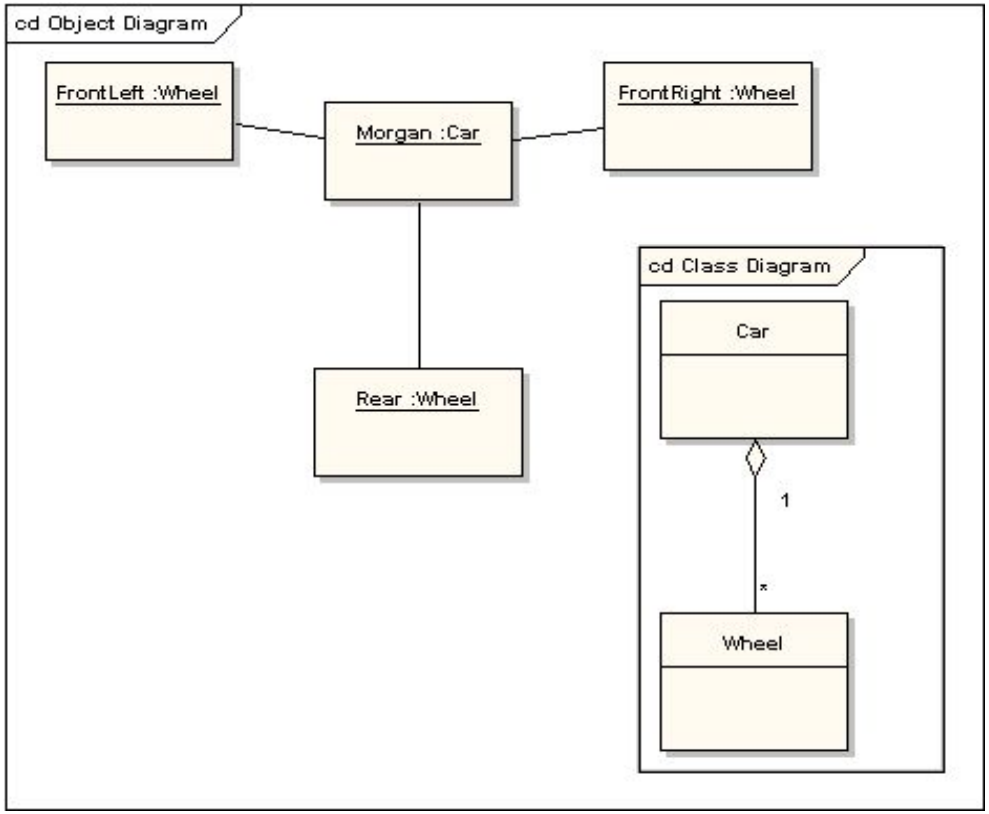
Statički UML dijagrami

- Dijagram klasa (*Class Diagram*)
  - prikazuje logičku strukturu apstrakcija: skup klasa, interfejsa, kolaboracija i njihovih relacija
- Dijagram objekata (*Object Diagram*)
  - prikazuje logičku strukturu instanci: skup objekata (instanci klasa) i njihovih veza
- Dijagram komponenata (*Component Diagram*)
  - Prikazuje fizičku organizaciju i zavisnosti između skupa komponenata
- Dijagram raspoređivanja (*Deployment Diagram*)
  - Prikazuje konfiguraciju čvorova obrade i komponenata koje su razmeštene na njima
- Dijagram paketa (*Package Diagram*) (UML 2.*)
  - Prikazuje statičku strukturu grupisanja elemenata modela u pakete
- Dijagram složene strukture (*Composite Structure Diagram*) (UML 2.*)
  - Prikazuje hijerarhijsko razlaganje primerka klase, komponente ili saradnje na delove

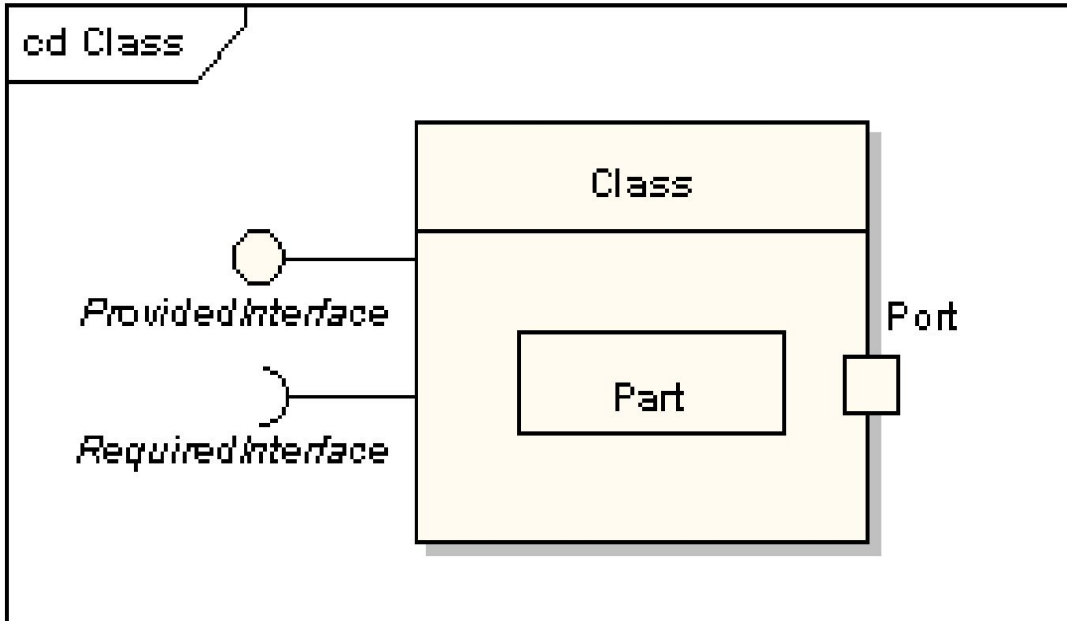
Class Diagram



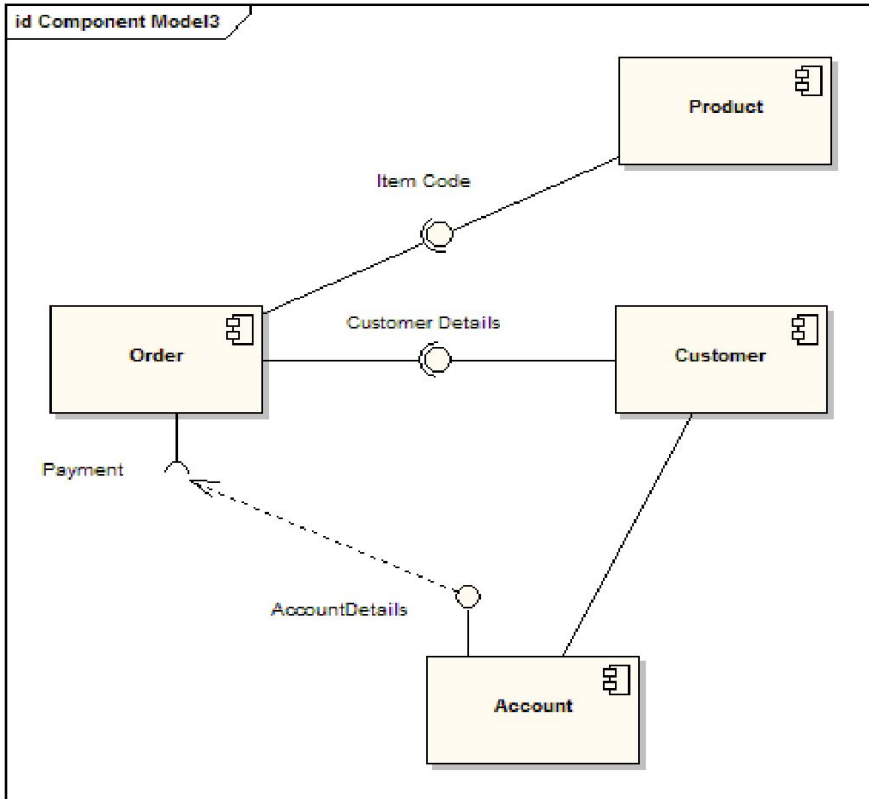
Object Diagram



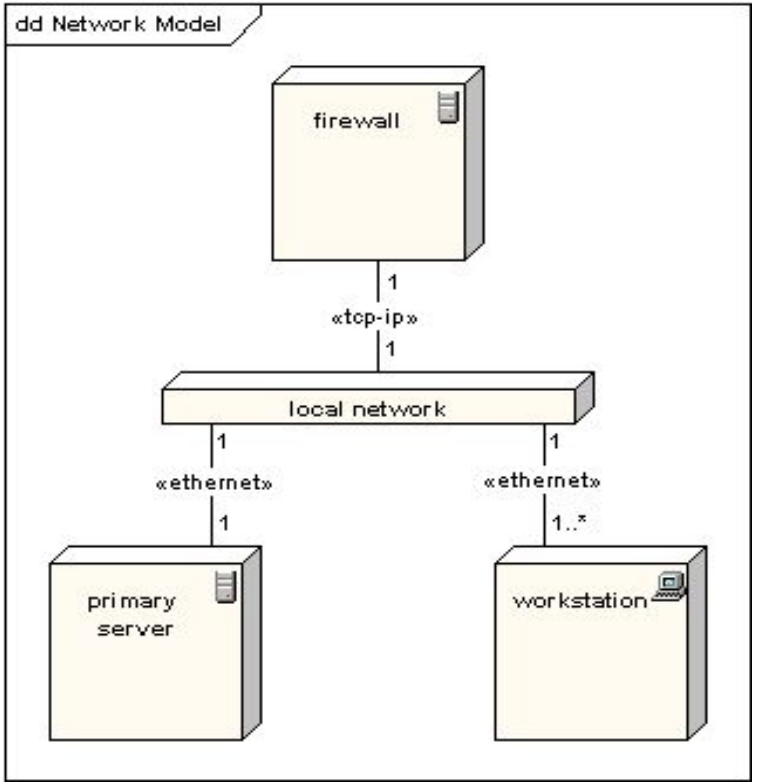
Composite Structure Diagram



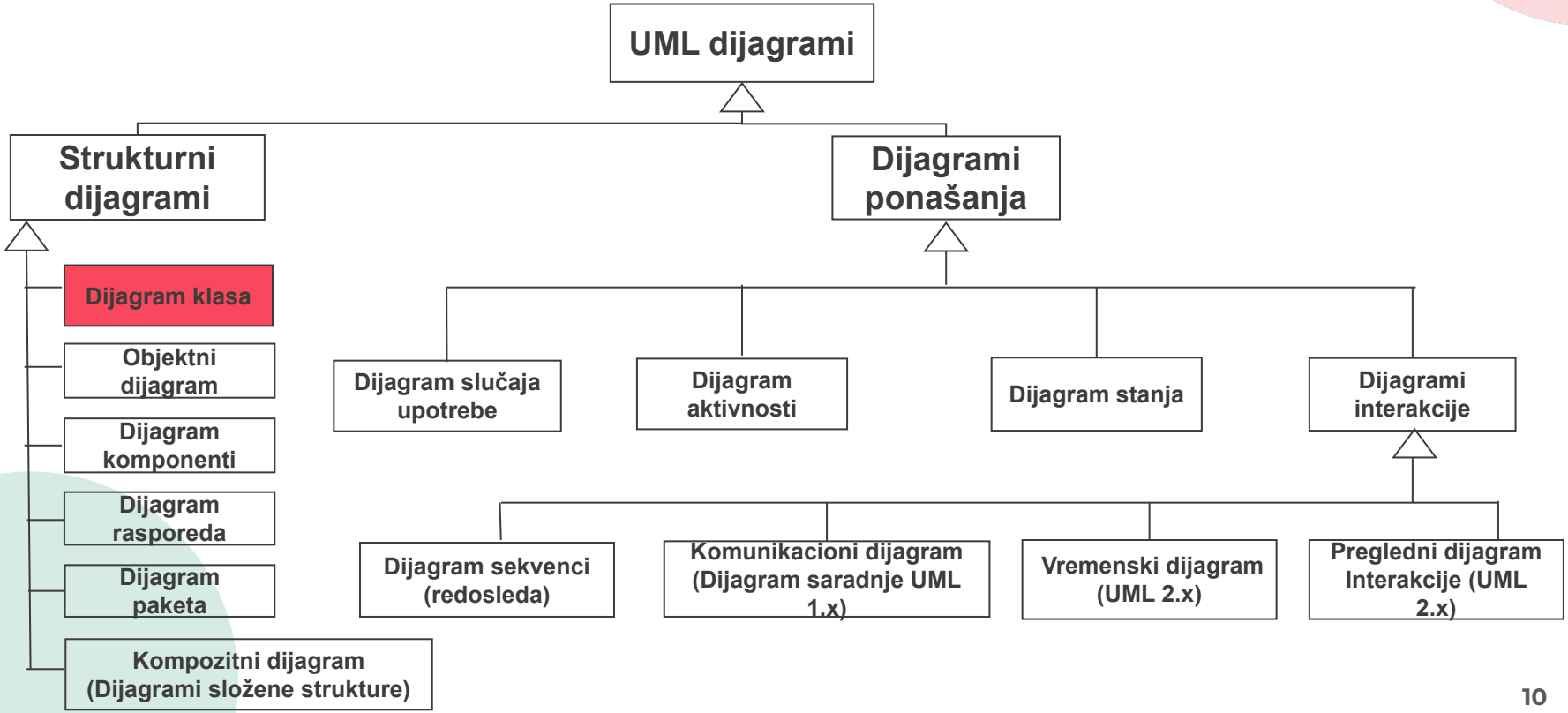
Component Diagram



Deployment Diagram



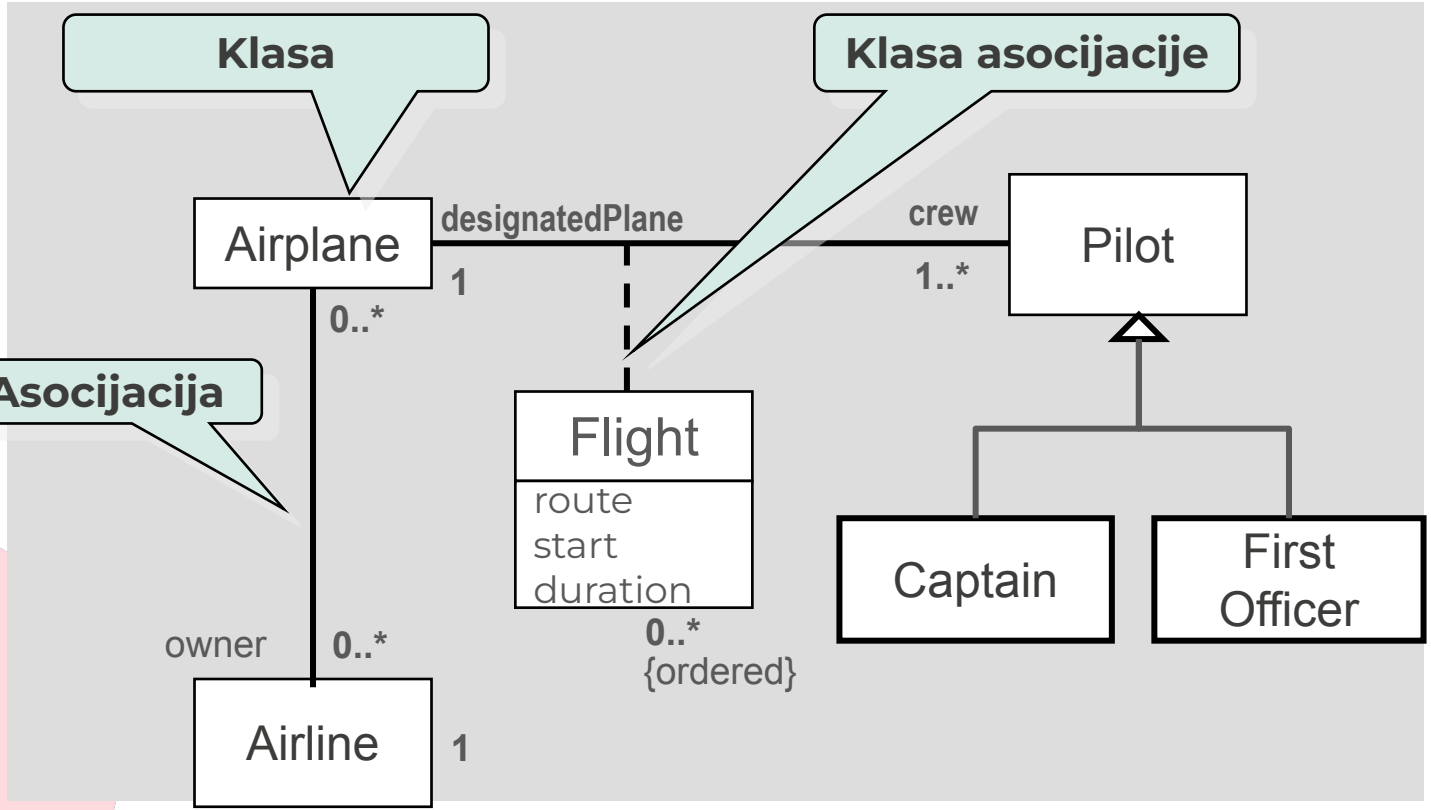
Koncepti funkcionalnog modelovanja ponašanja



Dijagrami klasa

- **Namena**
  - vizuelno iskazivanje modela statičke strukture sistema
    - šeme baze podataka i podšema
      - perzistentnih struktura podataka (zapisanih na eksternim memorijskim uređajima)
      - privremenih struktura podataka (koje mogu da egzistiraju u radnoj zoni programa, u operativnoj memoriji)

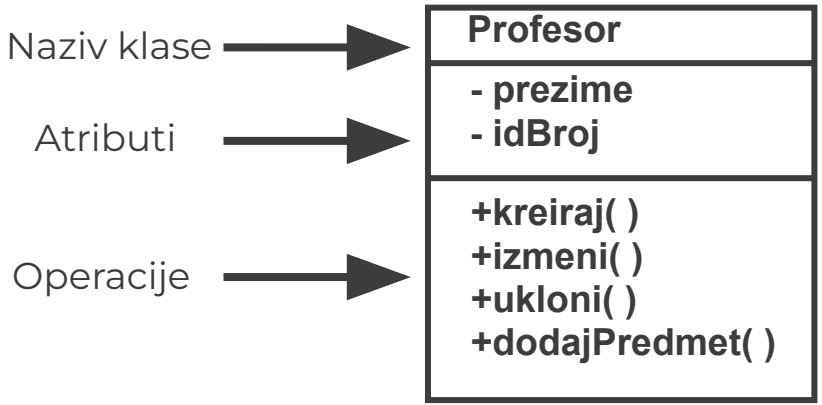
Dijagrami klasa



Dijagrami klasa

- UML 2.0 definicija: klasa je opis (model) skupa objekata koji dele istu specifikaciju karakteristika (*features*), ograničenja (*constraints*) i semantiku
  - karakteristike: atributi i operacije

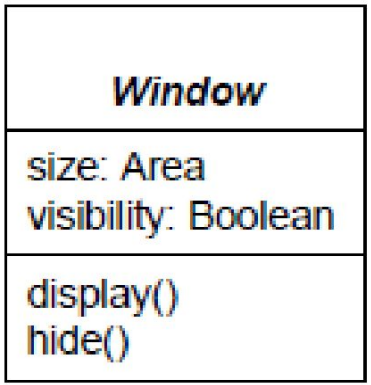
- Dijagramska notacija
  - odeljak za deklaraciju naziva i osobina klase
  - odeljak za deklaraciju atributa klase
  - odeljak za deklaraciju operacija klase



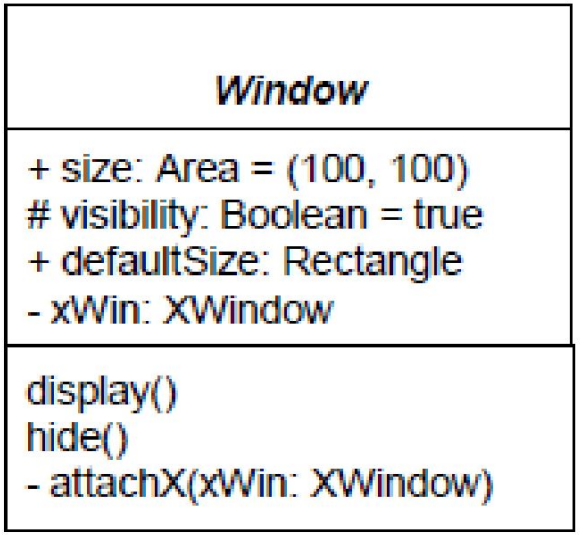
Dijagram klasa u različitim fazama razvoja softvera



Faza analize



Faza dizajna

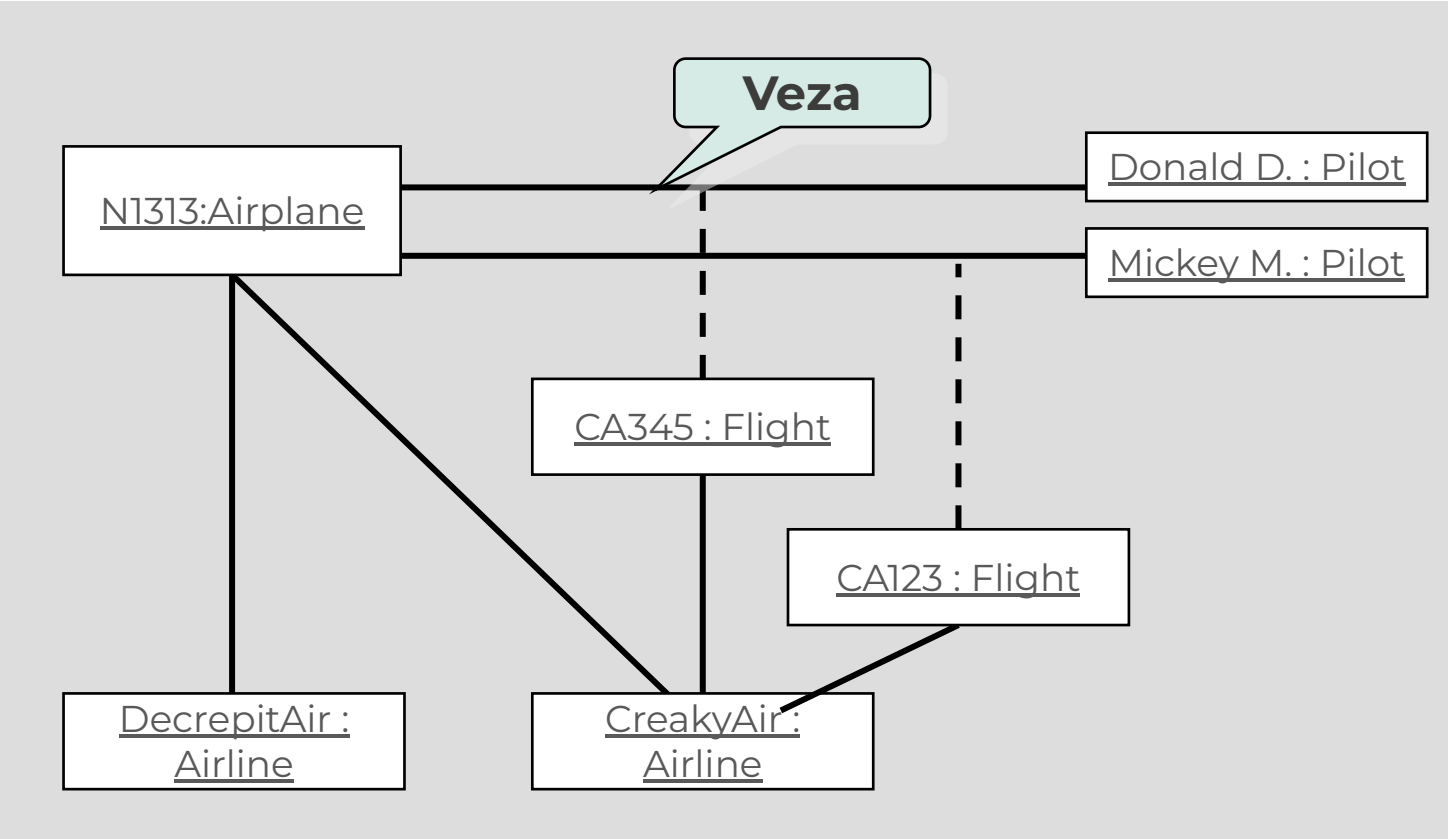


Faza implementacije

Dijagrami objekata

- **Namena**
  - vizuelno iskazivanje mogućeg, konkretnog stanja sistema u celini, ili njegovog dela
    - strukture podataka koja reprezentuje pojavu nad bazom podataka ili podšemom
    - bilo kakve strukture podataka privremenog karaktera, koja se nalazi u radnoj zoni programa, u operativnoj memoriji

Dijagrami objekata



Dijagrami objekata

- Dijagramska notacija
  - bilo koji, jedan objekat klase



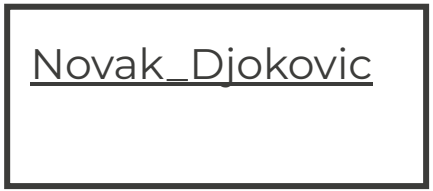
Samo ime klase

  - jedan određeni objekat klase



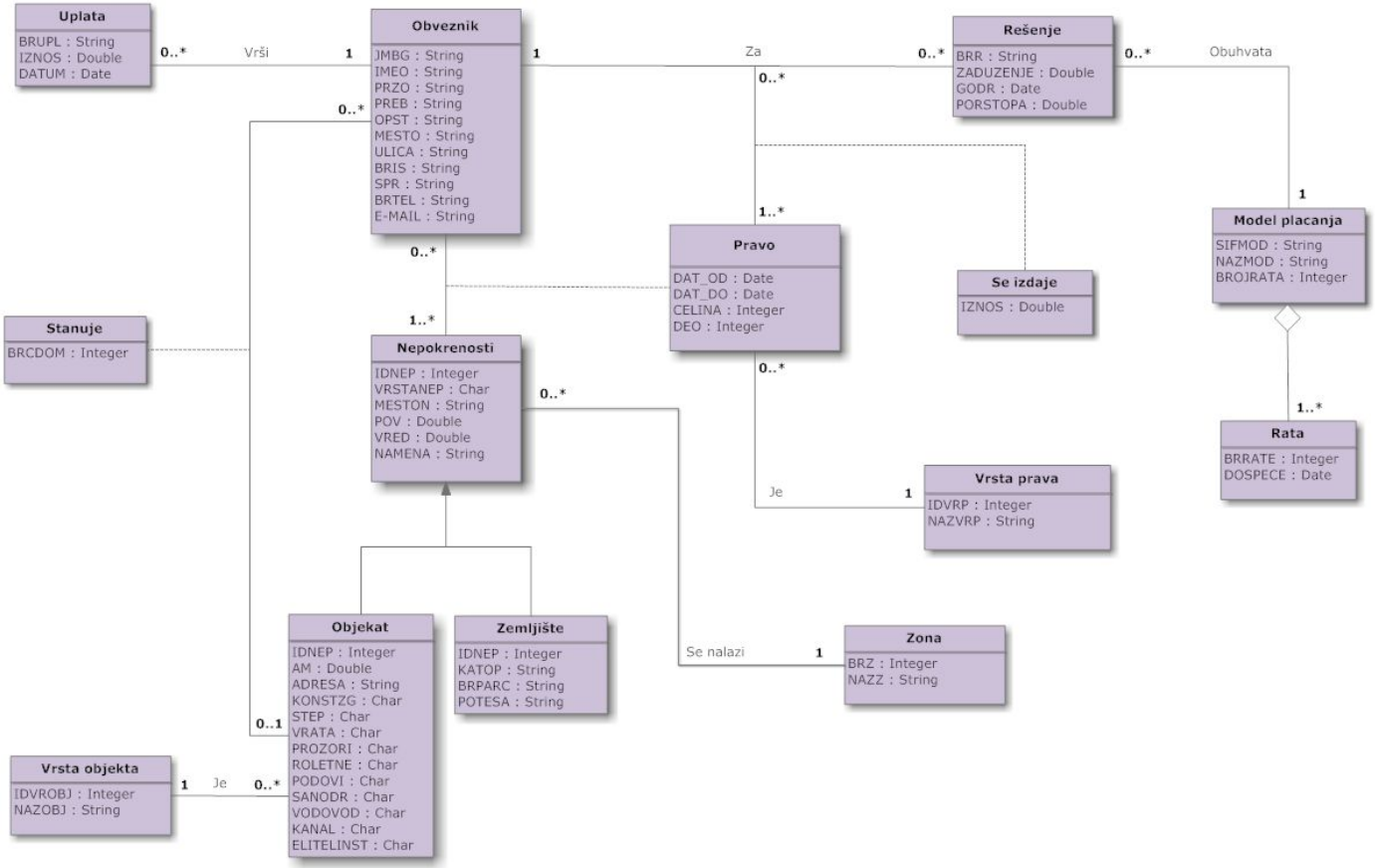
Ime klase i objekta

  - jedan, određeni objekat
    (klasa je "podrazumevana")

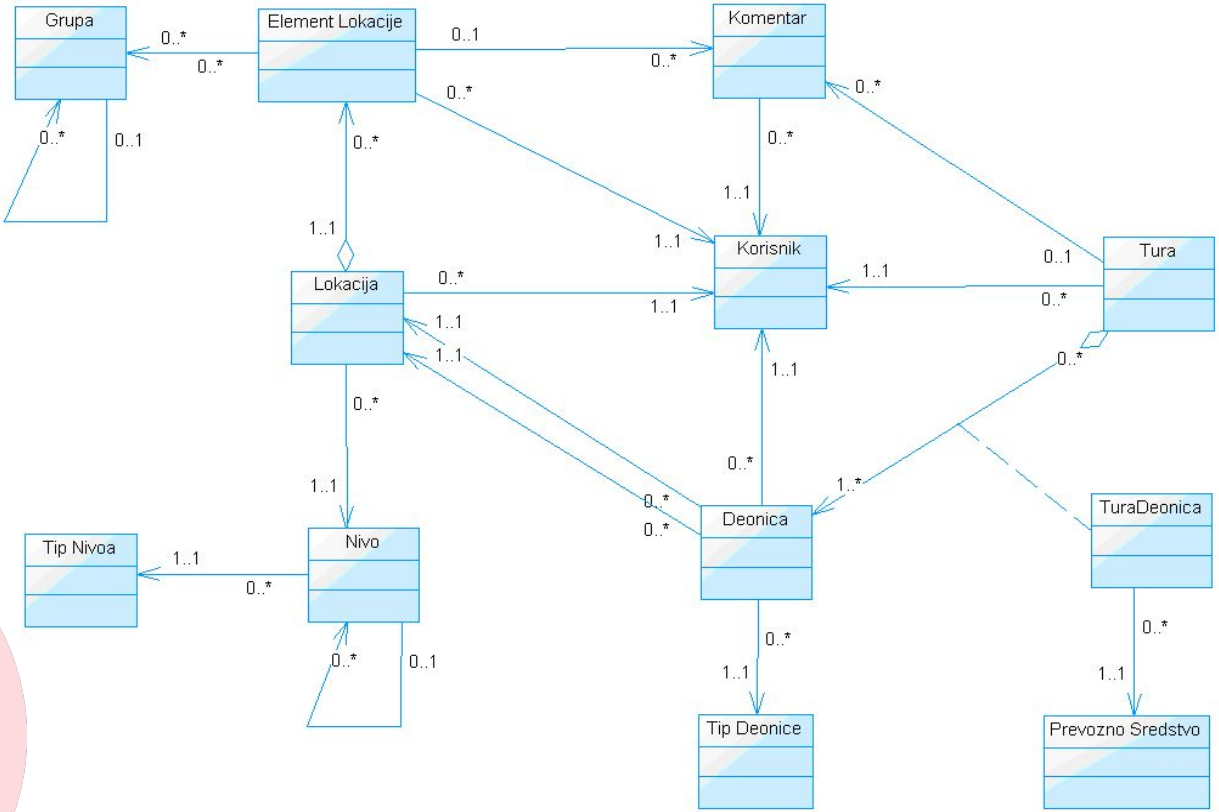


Samo ime objekta

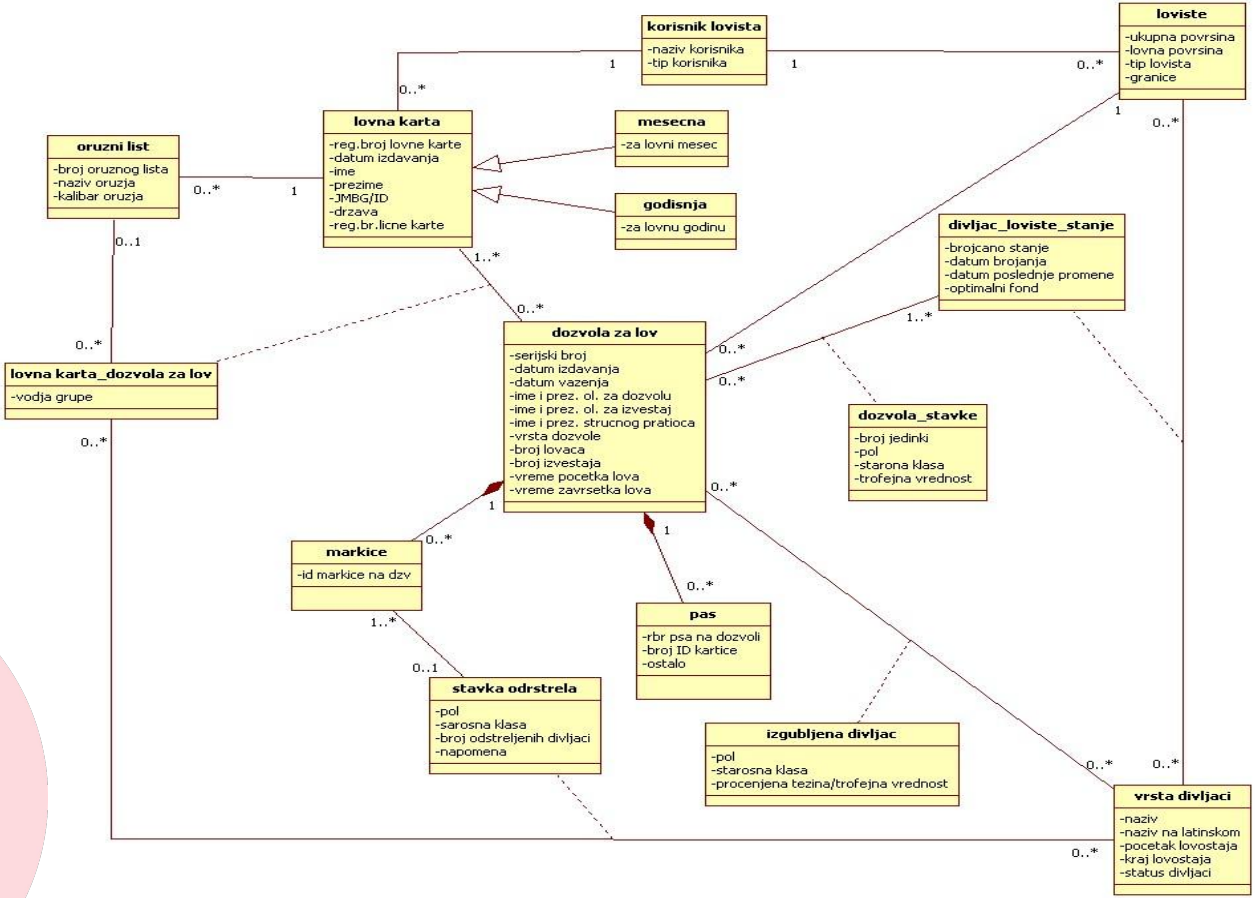
Primer dijagrama klase



Primer dijagrama klase

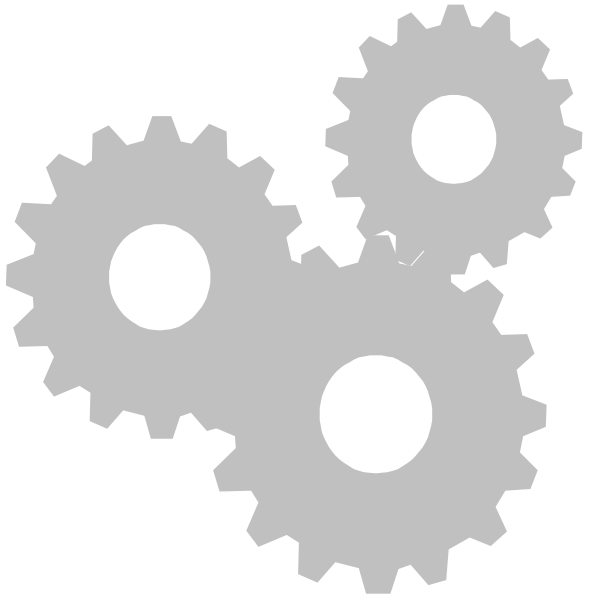


Primer dijagrama klase



Veza

- **Osnovne vrste veza**
  - asocijacija
    - agregacija
    - kompozicija
  - zavisnost
  - realizacija
  - generalizacija

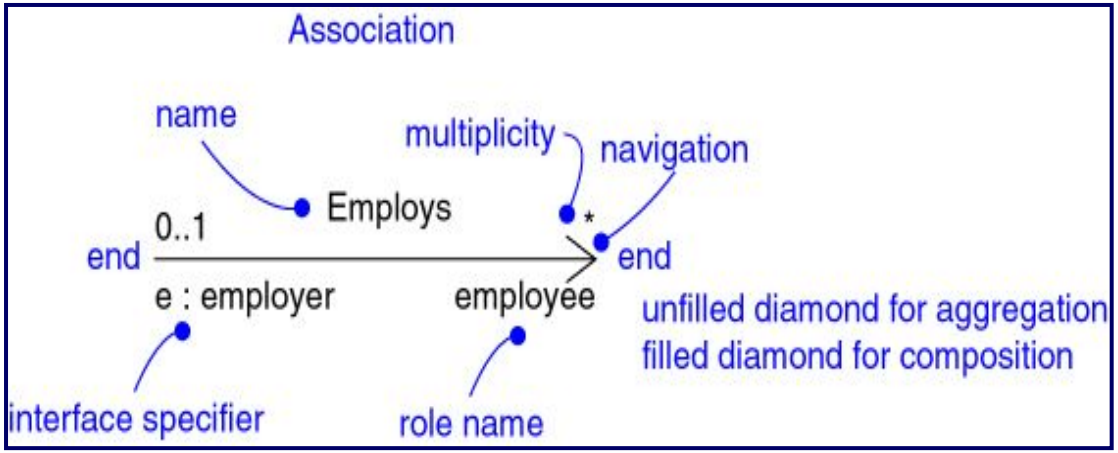


Veza

- **Asocijacija (Association)**
  - modelira strukturalne veze (odnose) između objekata različitih ili istih klasa
    - takve da predstavljaju osobine objekata povezanih klasa
    - koje se mogu iskazati putem vrednosti atributa, tj. stanja objekata

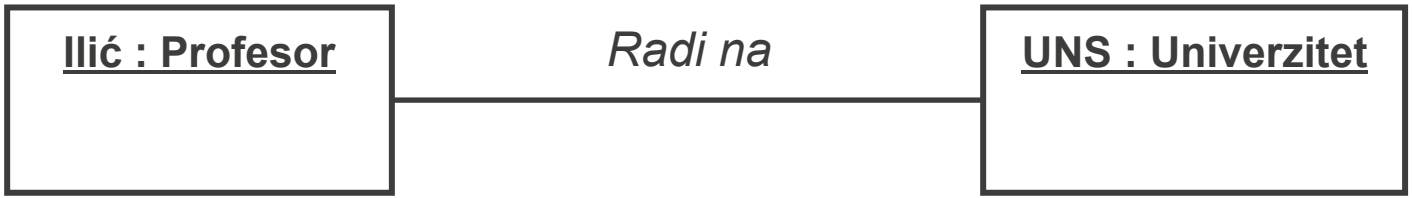
Veza

- **Asocijacija**
  - osnovne karakteristike
    - naziv i red
    - naznake uloga povezanih klasa
    - kardinalnosti preslikavanja skupova objekata
    - navigacija
    - kvalifikatori

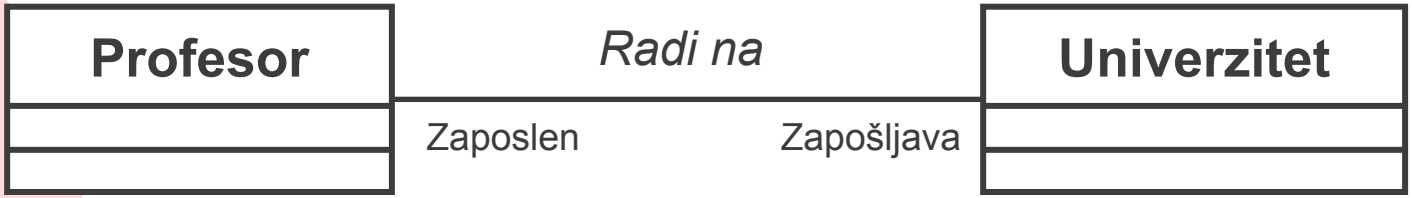


Veza

- **Dijagramska notacija**
  - asocijacija objekata

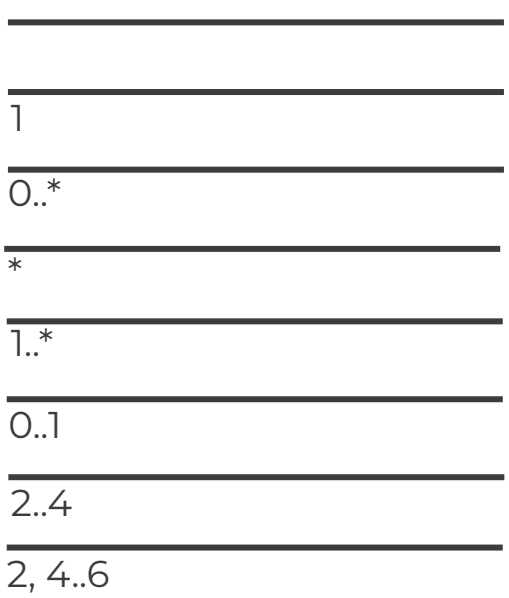


  - asocijacija klasa



Veza

- **Kardinalnost asocijacije**
  - neodređena
  - tačno jedan
  - nula ili više
  - nula ili više
  - jedan ili više
  - nula ili jedan
  - intervalna
  - nabrojana



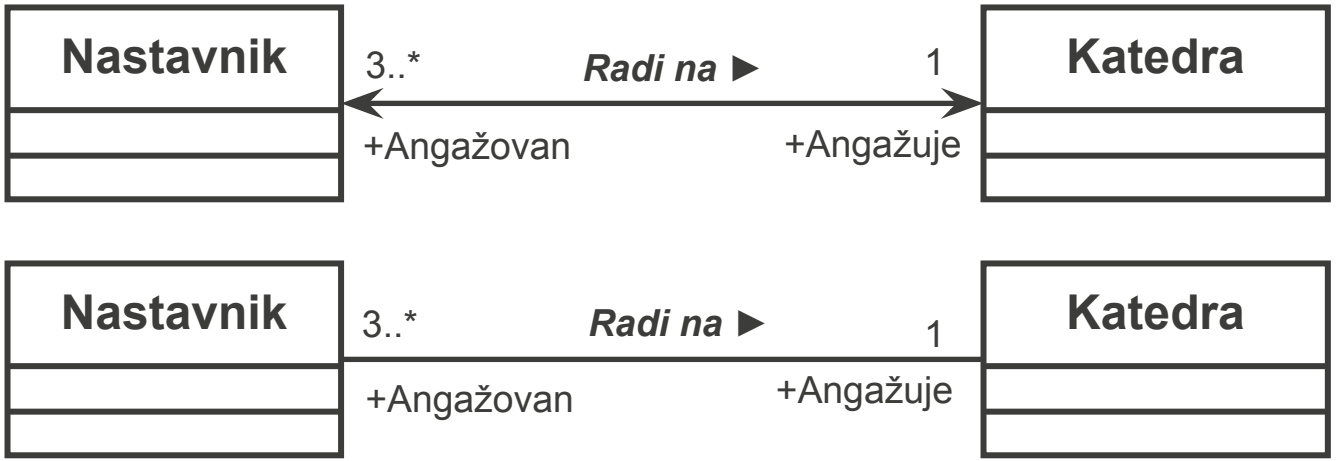
- UML 2.0 ne dopušta izraze sa diskontinuitetom vrednosti

Veza

- **Asocijacija**
  - primer
    - Klase: *Nastavnik* i *Katedra*
    - Asocijacija: *Radi*, reda ***n = 2* (binarna)**
      - Uloge i kardinalnosti:
        - *nastavnik **je angažovan** **na tačno jednoj** katedri*
        - *svaka katedra **angažuje** **barem tri** nastavnika*
      - Navigacija:
        - objekat klase nastavnik sadrži podatak o katedri na kojoj nastavnik radi
        - objekat klase katedra sadrži skup nastavnika koji na katedri rade

Veza

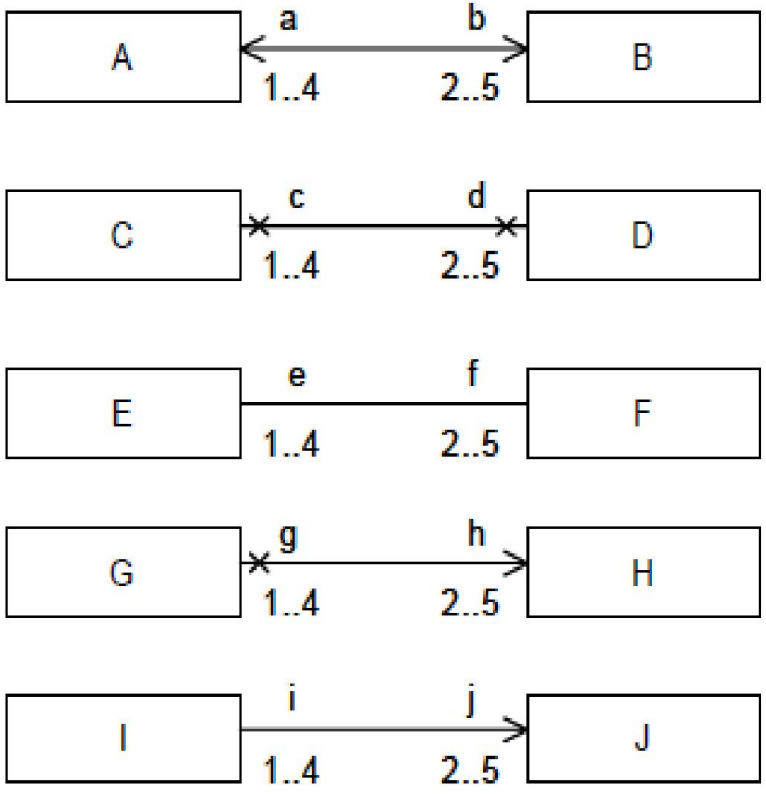
- **Asocijacija**
  - primer
    - Klase: *Nastavnik* i *Katedra*
    - Asocijacija: *Radi*, reda ***n = 2* (binarna)**



Veza

- Pravo pristupa preko asocijacije
  - Ograničava vidljivost (*visibility*) objekata u asocijaciji za “spoljašnji svet”
  - Označava se sa +, #, -, ~ ispred imena uloge odgovarajuće strane relacije
    - **+** znači da objektima sa te strane mogu pristupati svi objekti preko objekata klase sa druge strane veze
    - **-** znači da objektima klase sa te strane mogu pristupati samo objekti klase sa druge strane veze
    - **#** znači da objektima sa te strane veze mogu pristupati i objekti klasa izvedenih iz klase sa drugog kraja veze
    - **~** znači da objektima sa te strane veze mogu pristupati i objekti klasa iz istog paketa kao klasa sa drugog kraja veze

Navigacija veze



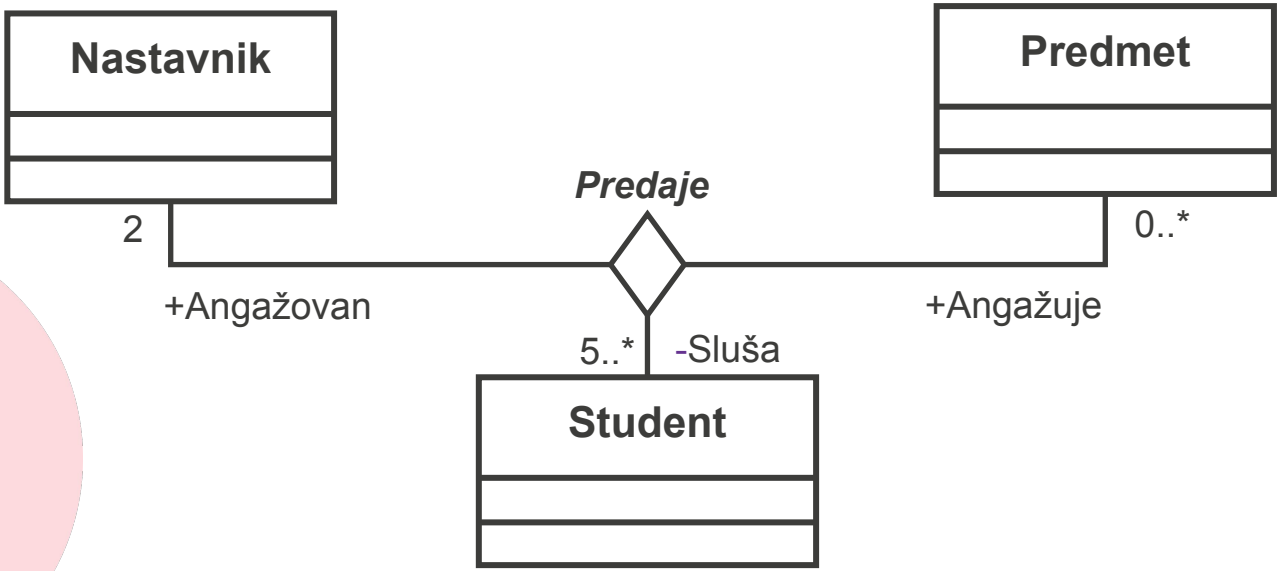
- Moguća navigacija u oba smera
- Nije omogućena navigacija ni u jednom smeru
- Nije specificirana mogućnost navigacije
- Moguća navigacija iz G u H, a nije moguća navigacija iz H u G
- Moguća navigacija iz I u J, a navigacija iz J u I nije specificirana

Asocijacija reda većeg od 2

- Primer
  - Klase: *Nastavnik*, *Predmet* i *Student*
  - Asocijacija: *Predaje*, reda ***n = 3* (ternarna)**
    - Uloge i kardinalnosti:
      - *jedan nastavnik predaje jedan predmet* ***za barem 5 studenata***
      - *jedan predmet, za jednog studenta, predaju* ***tačno dva nastavnika***
      - *jedan student, kod jednog nastavnika, može slušati* ***više predmeta***

Asocijacija reda većeg od 2

- Asocijacija: *Predaje*, reda ***n = 3*** **(ternarna)**
  - Nastavnik koji izvodi nastavu iz nekog predmeta može to da čini za najmanje 5 ili v studenata
  - Student nastavu iz nekog predmeta može da sluša kod tačno dva nastavnika
  - Nastavnik nekom studentu može da predaje više predmeta



Veza

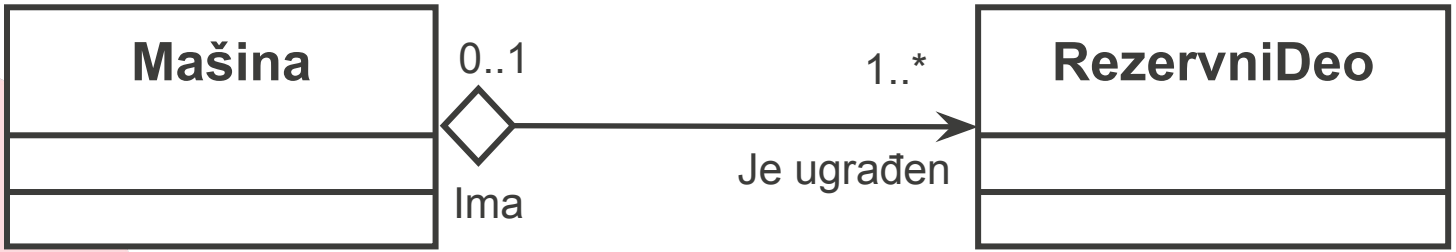
- **Agregacija (*Aggregation*)**
  - posebna vrsta asocijacije
  - modelira strukturalni odnos tipa **"celina-deo"**
    - jedan objekat, u nekom trenutku vremena, predstavlja sastavni deo drugog objekta
    - deo u agregaciji može biti zajednički deo **više** celina
  - Agregacija ne govori ništa o uzajamnom odnosu životnih vekova celine i dela

Veza

- **Agregacija**
  - primer
    - Klase: *Mašina* i *Rezervni_Deo*
    - Agregacija: *Ugrađuje_se*
      - *Rezervni deo se ugrađuje u tačno jednu mašinu i postaje njen deo*
      - *Rezervni deo se može, po potrebi, premestiti iz jedne u drugu mašinu*
      - *Rezervni deo predstavlja nezavisni sklop od mašine*

Veza

- **Agregacija**
  - primer
    - Klase: *Mašina* i *RezervniDeo*
    - Agregacija: *Ugrađuje_se*



Veza

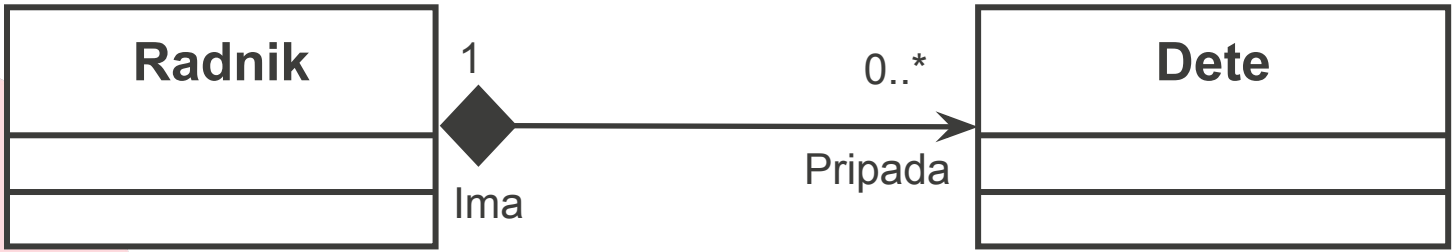
- **Kompozicija (*Composition*)**
  - posebna vrsta agregacije
    - jedan, zavisni objekat predstavlja sastavni deo drugog, nezavisnog objekta
    - životni vek zavisnog objekata (objekta tipa"deo") je **ograničen životnim vekom** nezavisnog objekta (objekta tipa "celina")
  - agregacija sa strogim vlasništvom
    - deo može nastati u toku života celine i može biti uništen pre uništenja celine
    - deo u kompoziciji može biti deo **samo jedne** celine

Veza

- **Kompozicija (*Composition*)**
  - primer
    - Klase *Radnik* i *Dete*
    - Kompozicija: *Ima*
      - *Zaposleni radnik u preduzeću može da obezbedi zdravstveno osiguranje i druga prava za svu svoju decu*
      - *Dete ostvaruje zdravstveno osiguranje i druga prava preko tačno jednog roditelja koji je radnik u preduzeću*
      - *Kada radnik napusti preduzeće, deca radnika gube mogućnost ostvarenja zdravstvenog osiguranja i drugih prava u tom preduzeću*

Veza

- **Kompozicija (*Composition*)**
  - primer
    - Klase *Radnik* i *Dete*
    - Kompozicija: *Ima*



Veza

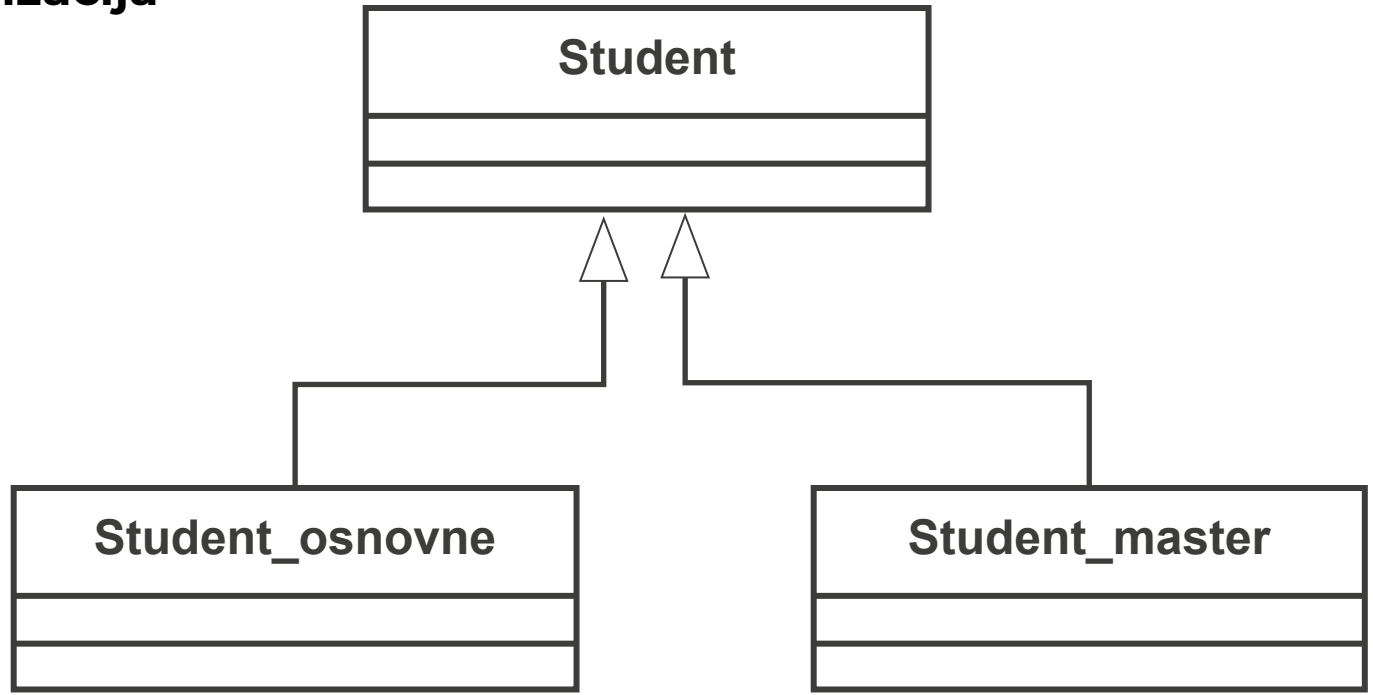
- **Generalizacija (*Generalization*)**
  - modelira odnos između klasa objekata u kojem
    - jedna klasa predstavlja specijalan slučaj druge klase
    - jedna klasa generalizuje svojstva drugih klasa
  - specijalizacija i generalizacija odnose se i na osobine i na ponašanje objekata klasa

Veza

- **Generalizacija**
  - primer
    - klasa *Student* generalizuje osobine klasa
      - *Student_osnovne*
      - *Student_master*
    - navedene klase
      - nasleđuju neke osobine i/ili neka ponašanja, definisana klasom *Student*
      - poseduju sopstvene osobine i/ili ponašanja, koje nisu karakteristične za klasu *Student*

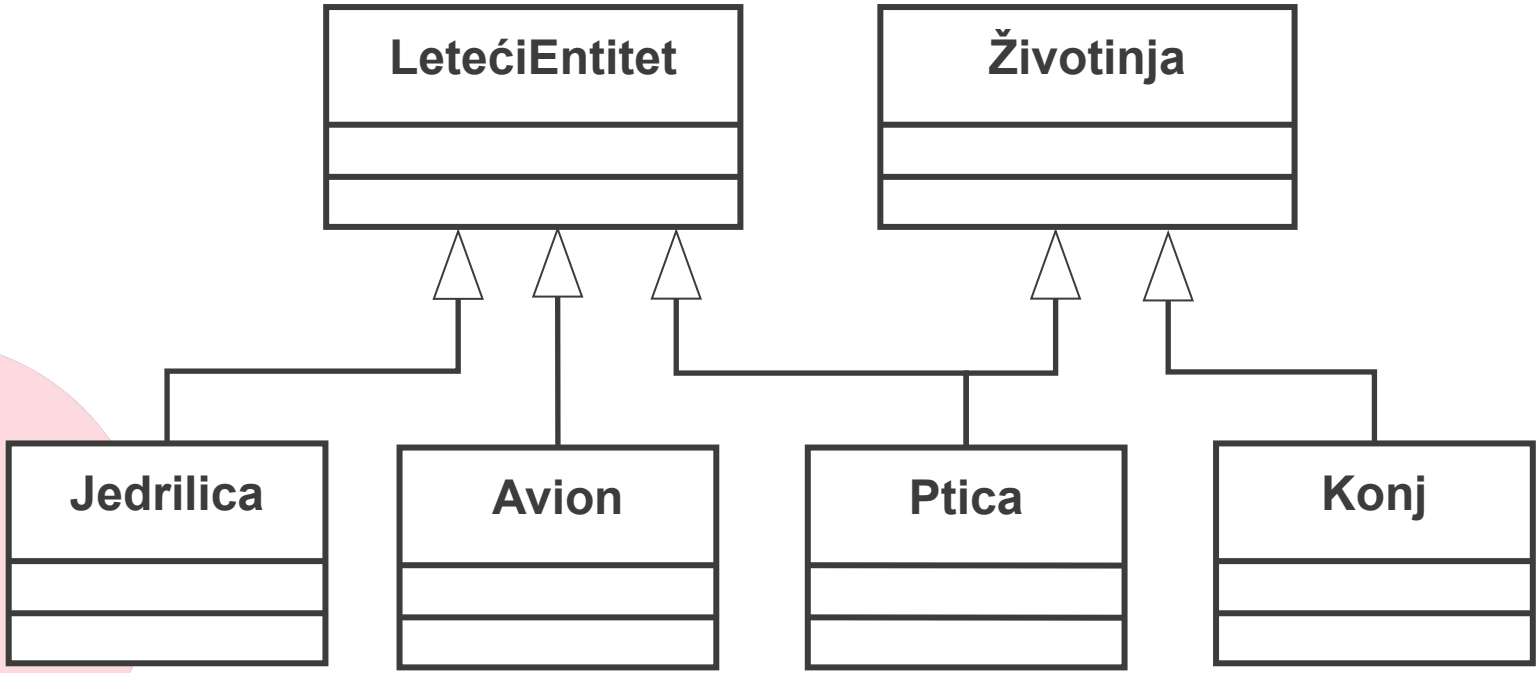
Veza

- **Generalizacija**



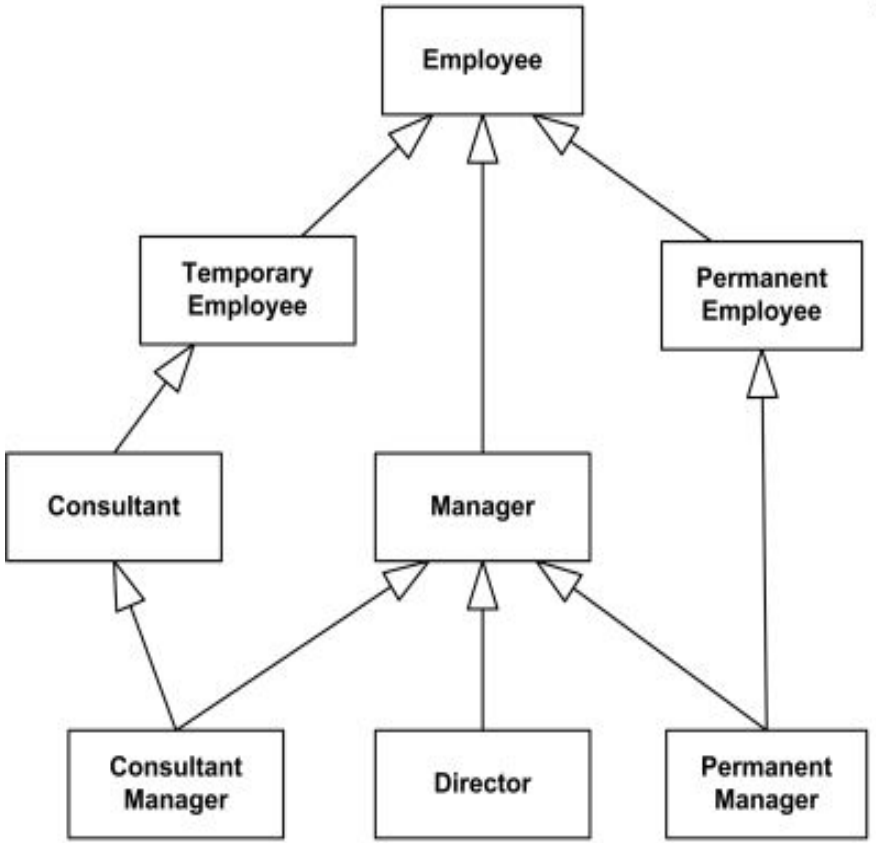
Veza

- **Generalizacija**
  - primer višestrukog nasleđivanja

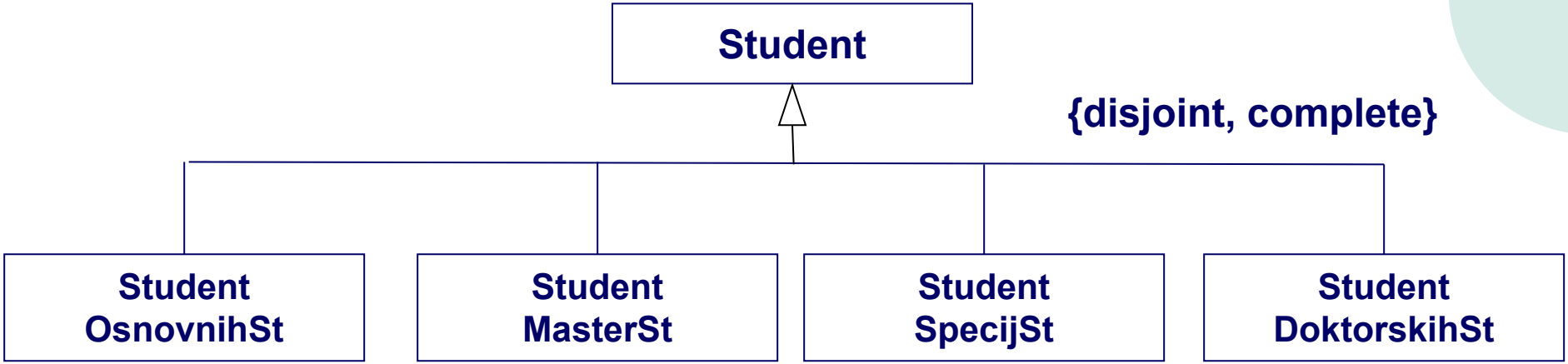


Veza

- **Generalizacija**
  - primer višestrukog nasleđivanja



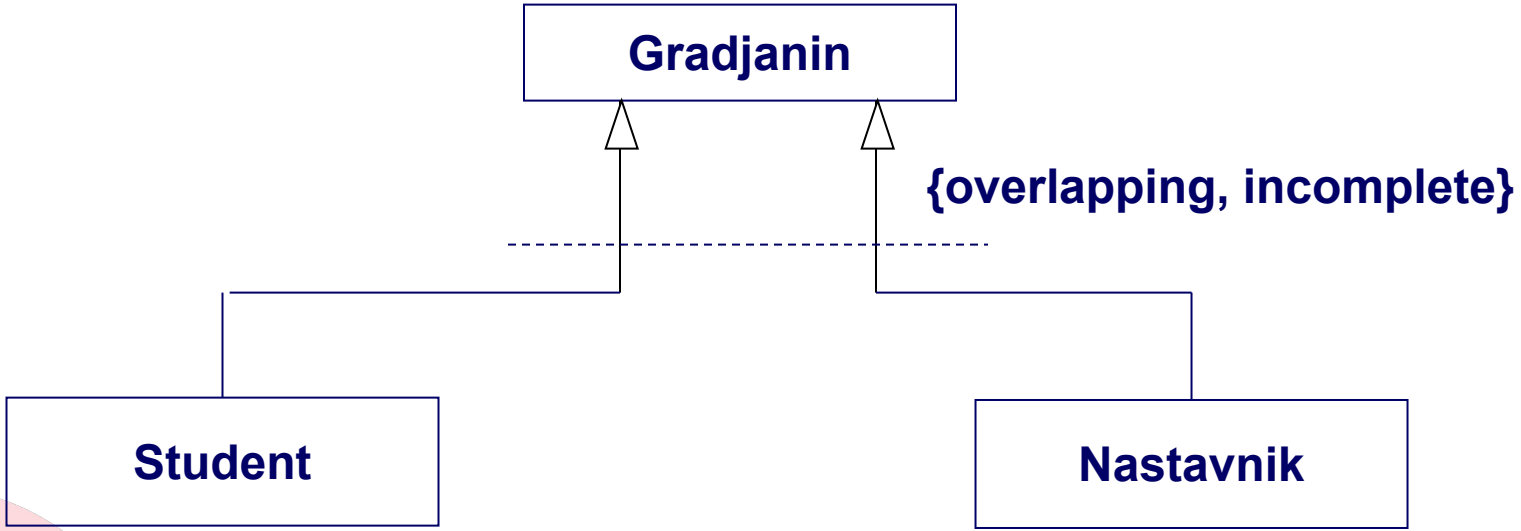
Primeri ograničenja u dijagramu klasa



Ovakav način specifikacije generalizacije znači da student ne može istovremeno biti, na primer, student osnovnih studija i student master studija (***disjoint***); kao i da ne postoji još neka "vrsta" studenata (***complete***)

overlapping vs. disjoint
complete vs. incomplete

Primeri ograničenja u dijagramu klasa



overlapping vs. disjoint
complete vs. incomplete

Dijagram klasa

- Pravo pristupa članu (vidljivost, *visibility*)
  - znak je napisan ispred atributa/operacije:
    - **+** javni član (podrazumevano)
    - **#** zaštićeni član
    - **~** paketski član
    - **-** privatni član

Poruka

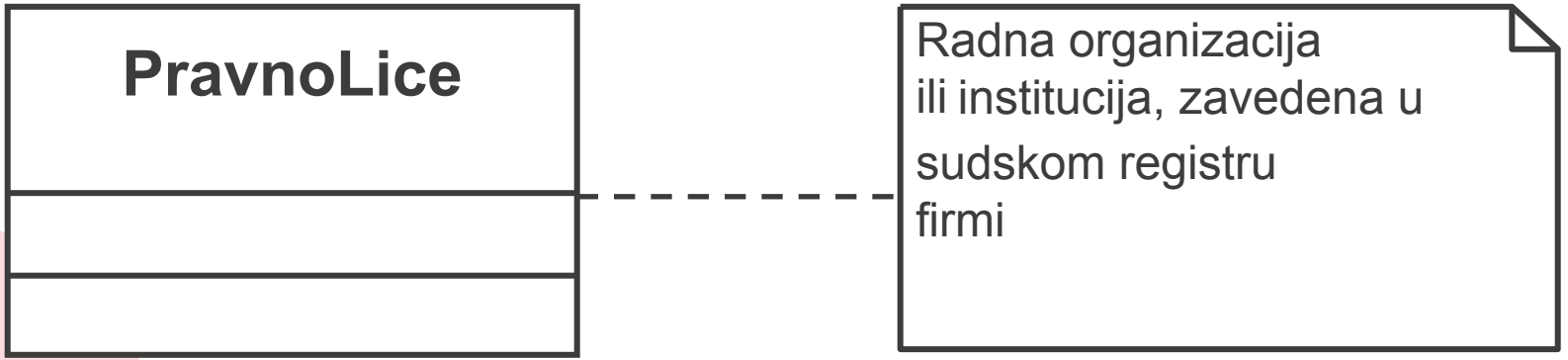
- Dijagramska notacija
  - **sinhrona poruka**
    - pošiljalac upućuje poruku primaocu i
    - čeka na povratni tok upravljanja, da bi nastavio započetu akciju (proceduru)
  - **asinhrona poruka**
    - pošiljalac upućuje poruku primaocu i
    - odmah nastavlja započetu akciju (proceduru)
  - **povratak toka upravljanja**
    - signal kojim pozvani objekat, po završetku operacije, označava predaju upravljanja tokom izvođenja aktivnosti pošiljaocu poruke

Napomena

- **Napomena (Komentar, *Note*)**
  - tekstualna informacija, pridodata nekom elementu modela
    - bliže objašnjava semantiku instance koncepta na koji se odnosi
    - koristi se za specificiranje uloga i ponašanja u modelu, kada za izražavanje semantike ne postoje odgovarajući koncepti višeg stepena formalizacije

Napomena

- **Napomena (Komentar, *Note*)**
  - Grafička notacija



Interfejs

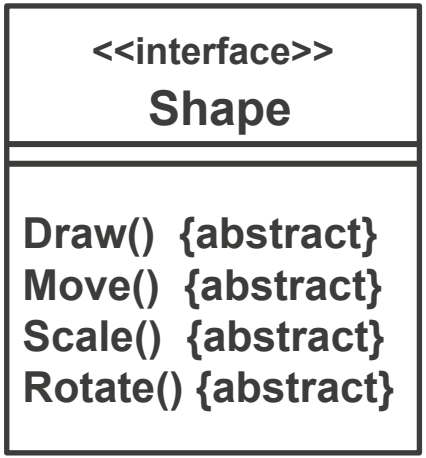
- **Interfejs (*Interface*)**
  - imenovani skup operacija (deklaracija metoda)

    Naziv({DekO1,..., DekOn})

  - u UML-u je to složeni koncept
    - izveden iz pojma klase
    - definisan putem ugrađenog stereotipa <<interface>>

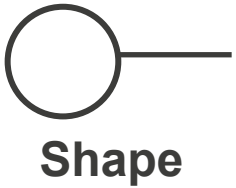
Interfejs

- **Grafička notacija**
  - klasični način predstavljanja

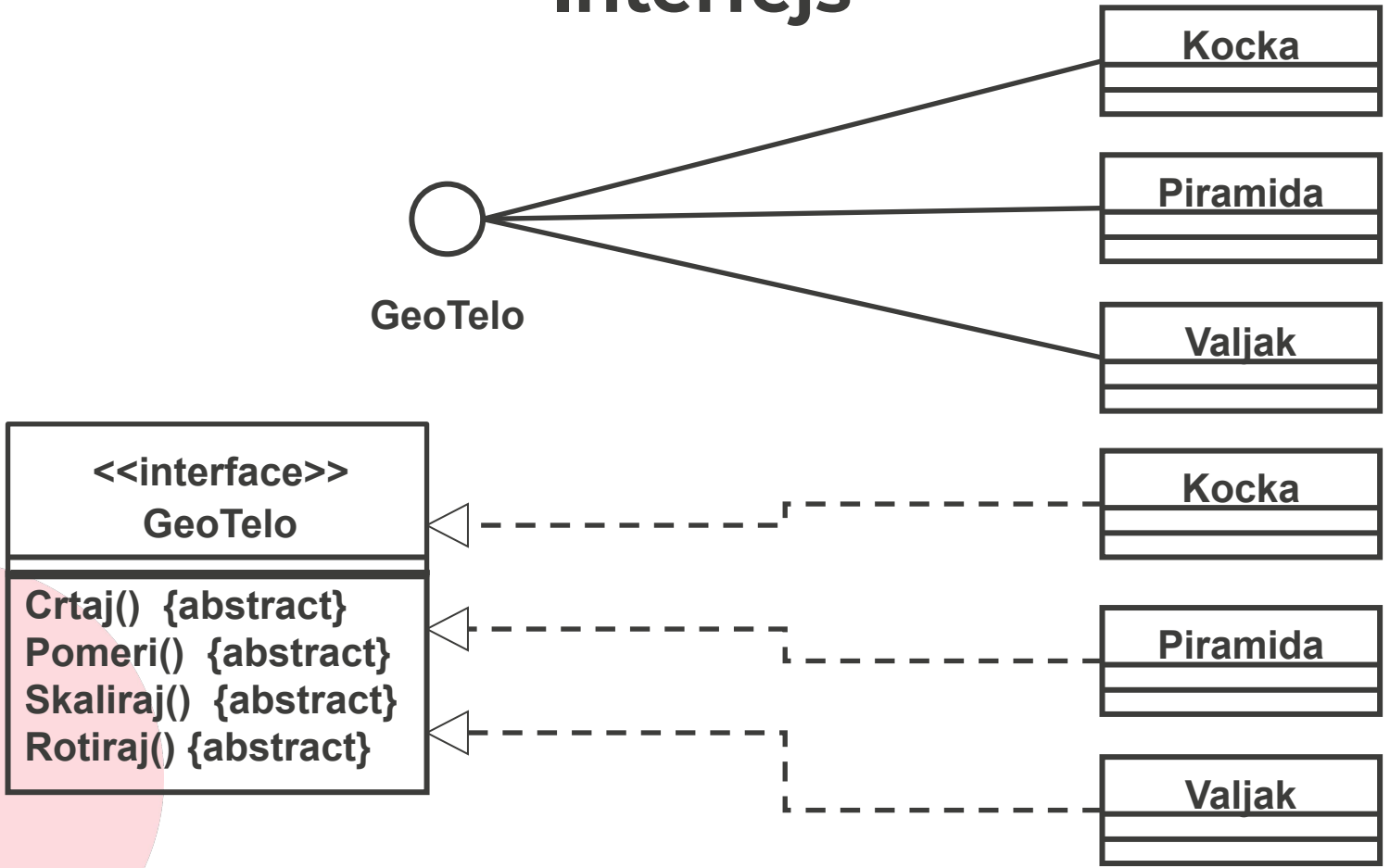


  - ikonični način predstavljanja ("lollipop")



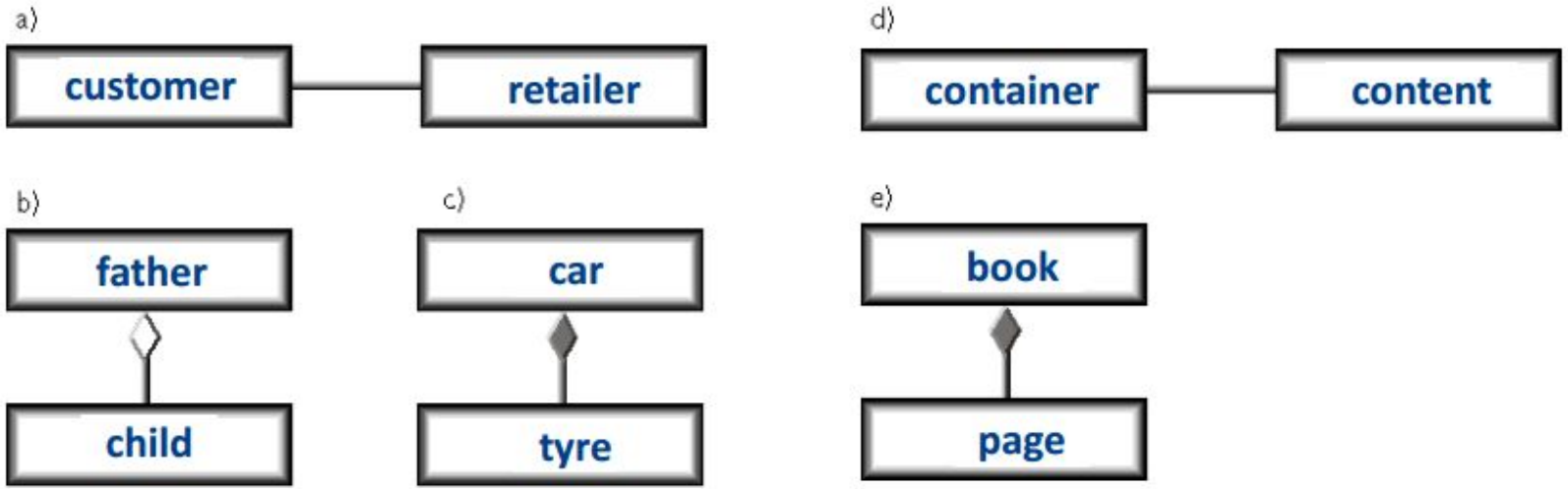


Interfejs


Kocka
Piramida
Valjak
GeoTelo
<<interface>>
GeoTelo
Crtaj() {abstract}
Pomeri() {abstract}
Skaliraj() {abstract}
Rotiraj() {abstract}
Kocka
Piramida
Valjak

Vežba 1

- Da li asocijacije na sledećim dijagramima imaju smisla? Ako ne, predložite bolje rešenje.



Vežba 2

- Identifikovati klase, atribute i metode i na osnovu datog opisa nacrtati dijagram klasa:

  Uzmimo za primer banku i njene klijente. Svaki klijent može otvoriti jedan ili više tekućih računa. Za svakog klijenta evidentiraju se podaci o imenu, prezimenu, adresi i datumu rođenja. Svaki klijent može i da zatvori bilo koji od svojih računa. Svi računi imaju stopu kamate na štednju. Svaki račun ima svoj jedinstveni broj. Korisnik može da uplati na račun ili podigne sa njega proizvoljnu sumu novca. Kako bi se izračunala kamata, za svaku transakciju na nekom računu beleži se datum transakcije, tip transakcije i suma.

- Prema nacrtanom dijagramu klasa, nacrtati dijagram objekata za korisnika Petra Petrovića iz Novog Sada, rođenog 05/05/1995, koji ima račun sa brojem 4435 i na njega je 11/10/2015 uplatio 1000 eura i 15/10/2015 podigao 300 eura.

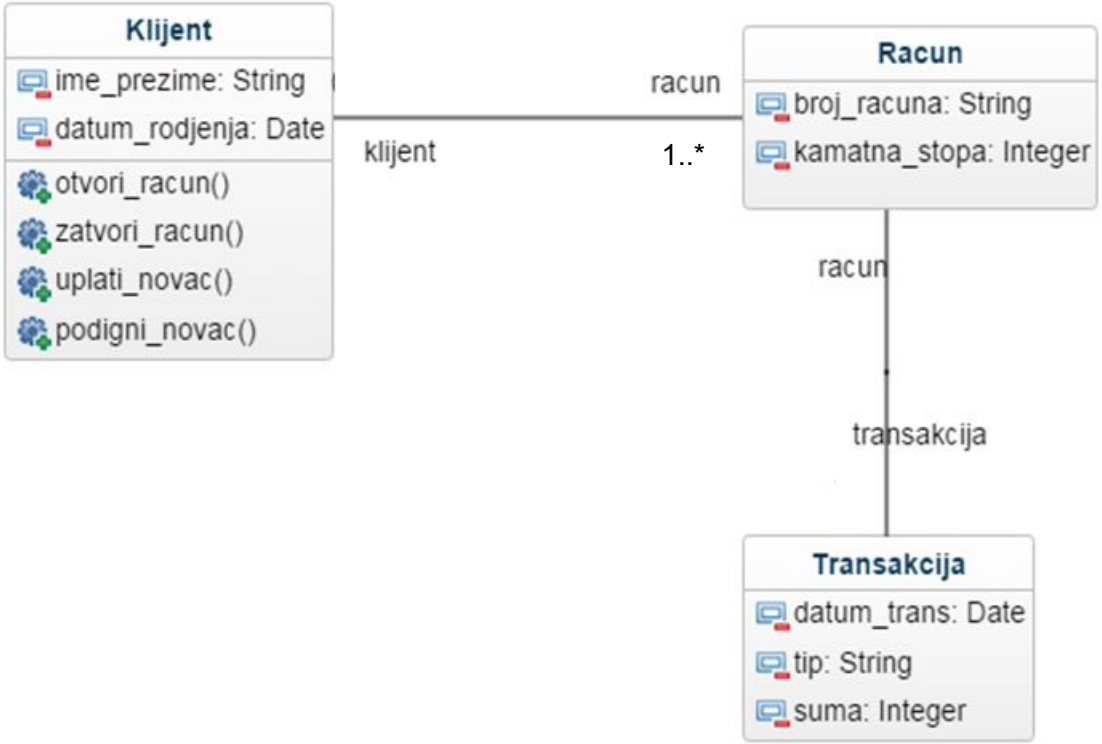
Vežba 3

- Nacrtati UML dijagram klasa koji reprezentuje sledeće elemente hokejaške lige. Svaka liga sastoji se od najmanje 4 hokejaška tima. Svaki tim sastoji se od 6 do 12 igrača i jednog igrača koji je kapiten. Tim mora imati ime i zapis o broju bodova na tabeli. Svaki igrač ima svoje ime i poziciju. Timovi između sebe igraju utakmice. Svaka utakmica ima rezultat i mesto odigravanja. Timovi su nekad predvođeni trenerom. Svaki trener ima nivo akreditacije i broj godina iskustva. Treneri i igrači su osobe, a svaka osoba ima ime i prezime.

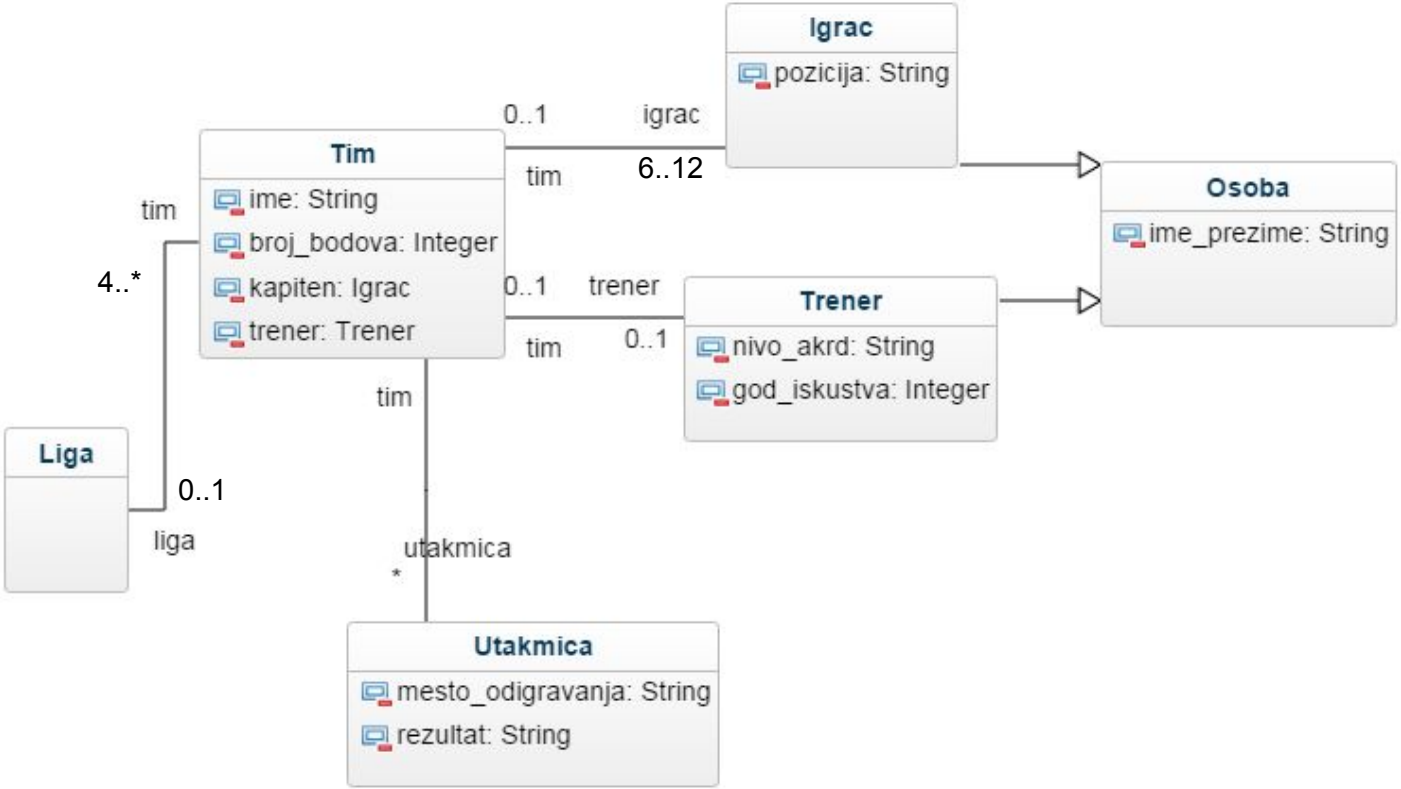
Rešenje 1

- a) jedno od rešenja
- b) kompozicija
- c) agregacija
- d) jedno od rešenja
- e) jedno od rešenja

Rešenje 2


Klijent
ime_prezime: String
datum_rodjenja: Date
otvori_racun()
zatvori_racun()
uplati_novac()
podigni_novac()
racun
klijent
1..*
Racun
broj_racuna: String
kamatna_stopa: Integer
racun
transakcija
Transakcija
datum_trans: Date
tip: String
suma: Integer

Rešenje 3


Igrac
pozicija: String
0..1
igrac
6..12
tim
Tim
ime: String
broj_bodova: Integer
kapiten: Igrac
trener: Trener
tim
4..*
Osoba
ime_prezime: String
0..1
trener
tim
0..1
Trener
nivo_akrd: String
god_iskustva: Integer
tim
Liga
0..1
liga
utakmica
*
Utakmica
mesto_odigravanja: String
rezultat: String


Hvala.